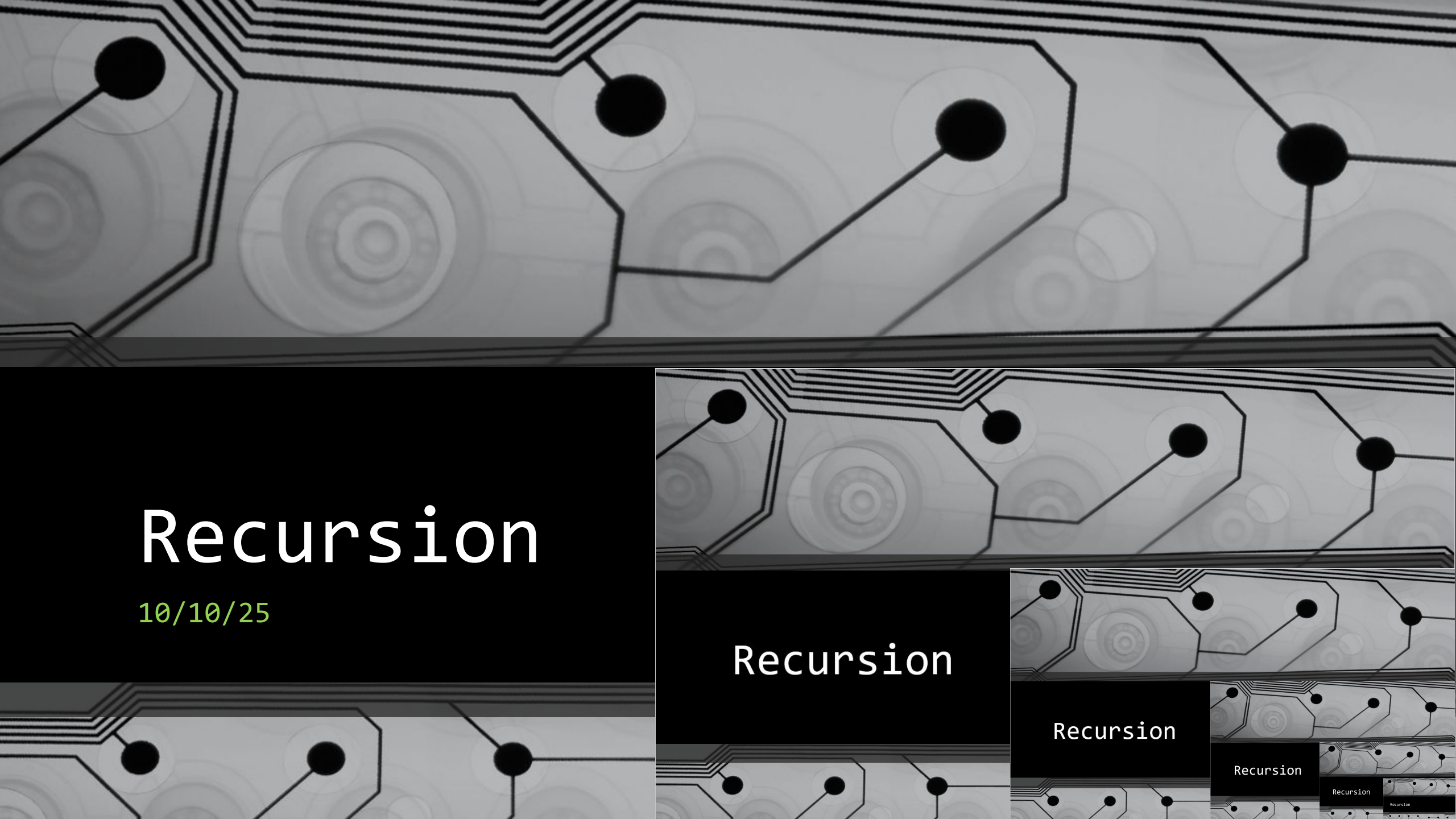




Recursion

10/10/25

Recursion

Recursion

Recursion

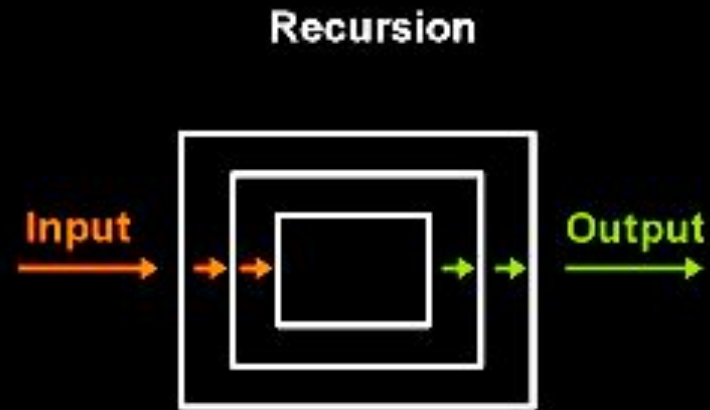
Recursion

Recursion

Recursion

recursion occurs when a thing is defined in terms of itself

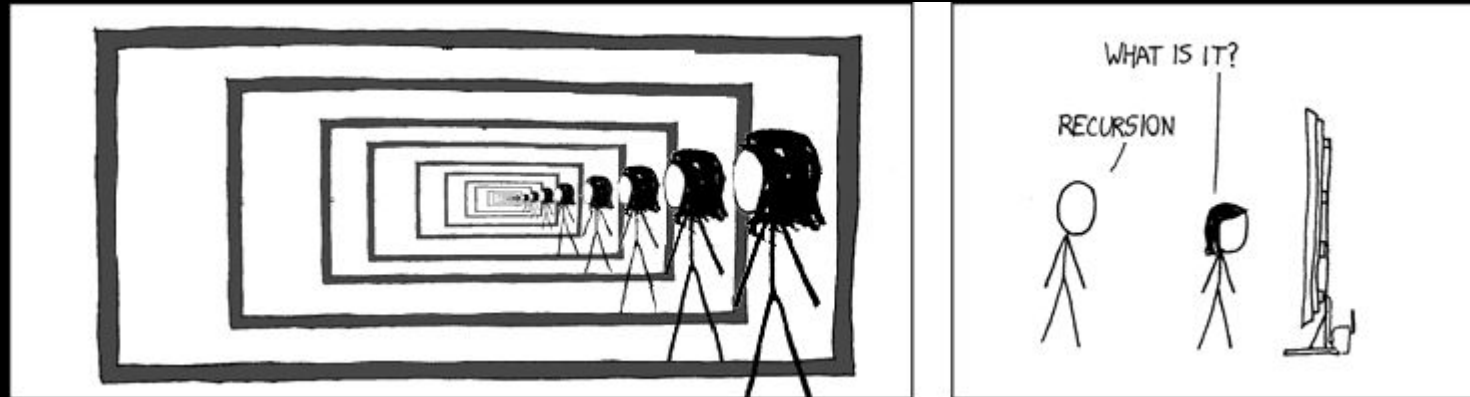
recursive function: is a function that calls itself with a smaller input than what it had originally received



Recursion is another way to break big problems into small problems

If a problem is small, solve it directly. (we call that a "**base case**", something that you immediately know the answer to)

Otherwise, break off a piece of the problem that you can figure out, and then try solving the smaller remaining portion (by breaking off another piece and so on, over and over... that is our "**recursive step**").



Recursion intuition: take one step toward the solution and ask your clone to handle the rest

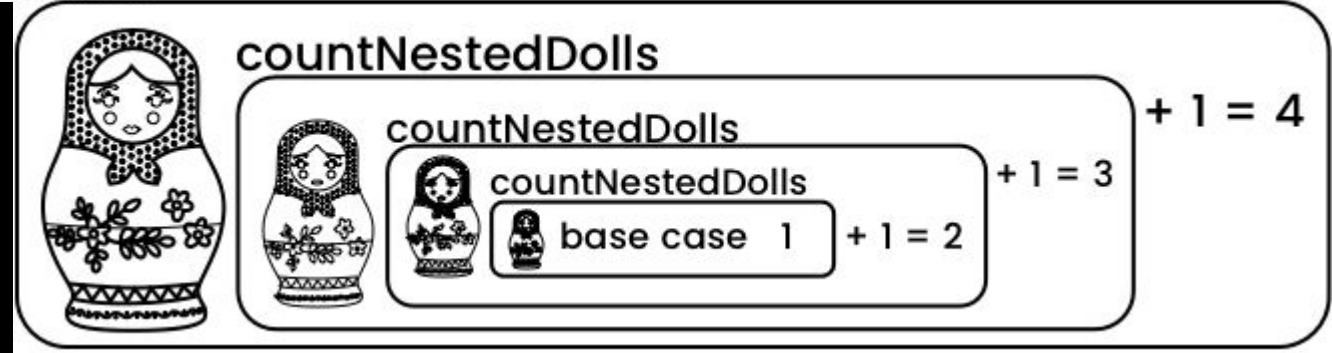


How many Russian nesting dolls do we have inside some large nesting doll?

No clue but 1 for this one + all the ones inside it!

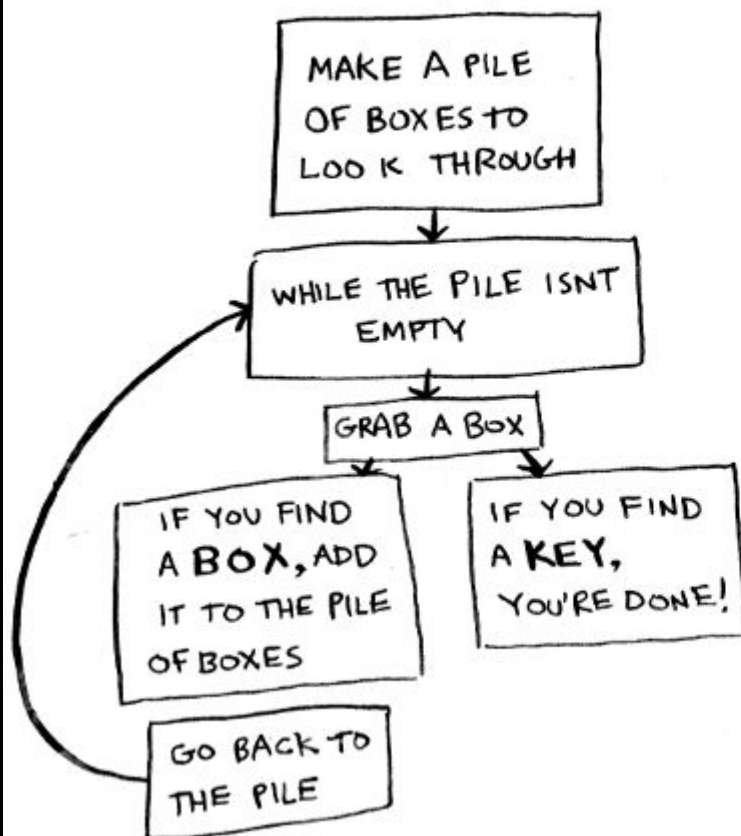
Keep opening dolls and adding one until we reach the tiny one that doesn't open ← Base Case of 1 doll.

count

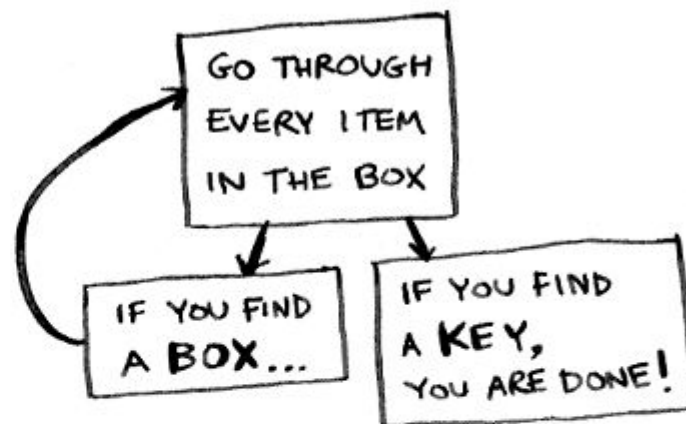


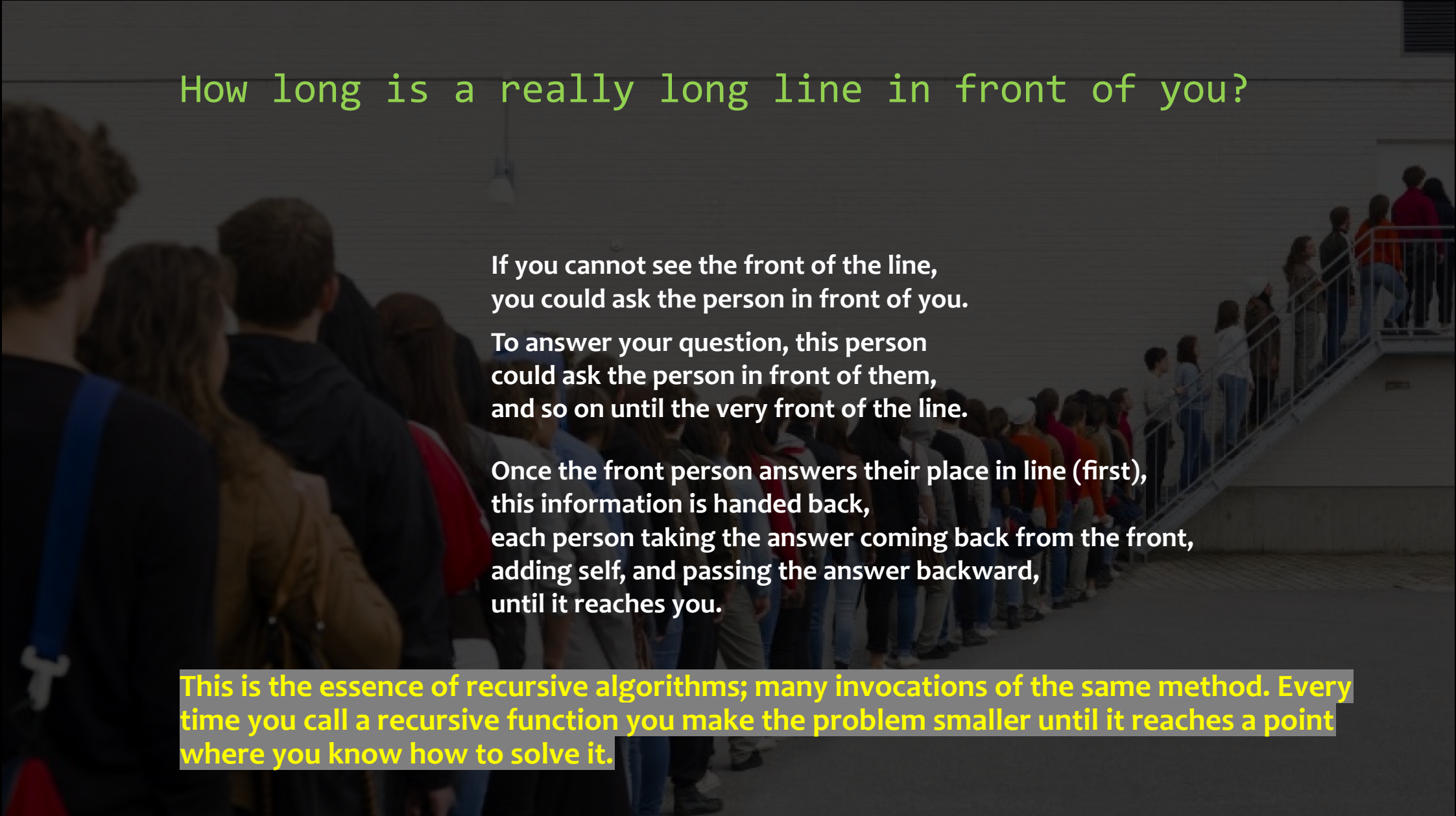
Iteration vs Recursion: two ways to "loop" through your code

Iterative Approach



Recursive Approach



A photograph of a long line of people standing outdoors, possibly on a campus. The line extends from the foreground into the distance, where it appears to be going up a set of stairs. The image is dimmed to serve as a background for text.

How long is a really long line in front of you?

If you cannot see the front of the line,
you could ask the person in front of you.

To answer your question, this person
could ask the person in front of them,
and so on until the very front of the line.

Once the front person answers their place in line (first),
this information is handed back,
each person taking the answer coming back from the front,
adding self, and passing the answer backward,
until it reaches you.

This is the essence of recursive algorithms; many invocations of the same method. Every time you call a recursive function you make the problem smaller until it reaches a point where you know how to solve it.

hmm.. I don't know
how long the line is,
but it's whatever the
person in front of me
says, plus 1 (me)



<https://www.inc.com/jill-krasny/why-waiting-in-lines-makes-things-more-appealing.html>

I don't know either, but
it's whatever the
person in front of me
says, plus me



<https://www.inc.com/jill-krasny/why-waiting-in-lines-makes-things-more-appealing.html>

no clue, but it's
whatever the
person in front of
me says, plus me



<https://www.inc.com/jill-krasny/why-waiting-in-lines-makes-things-more-appealing.html>

whatever they say
up ahead plus 1 for
me too!



<https://www.inc.com/jill-krasny/why-waiting-in-lines-makes-things-more-appealing.html>

I'm first! So my line is
of length 1 :D

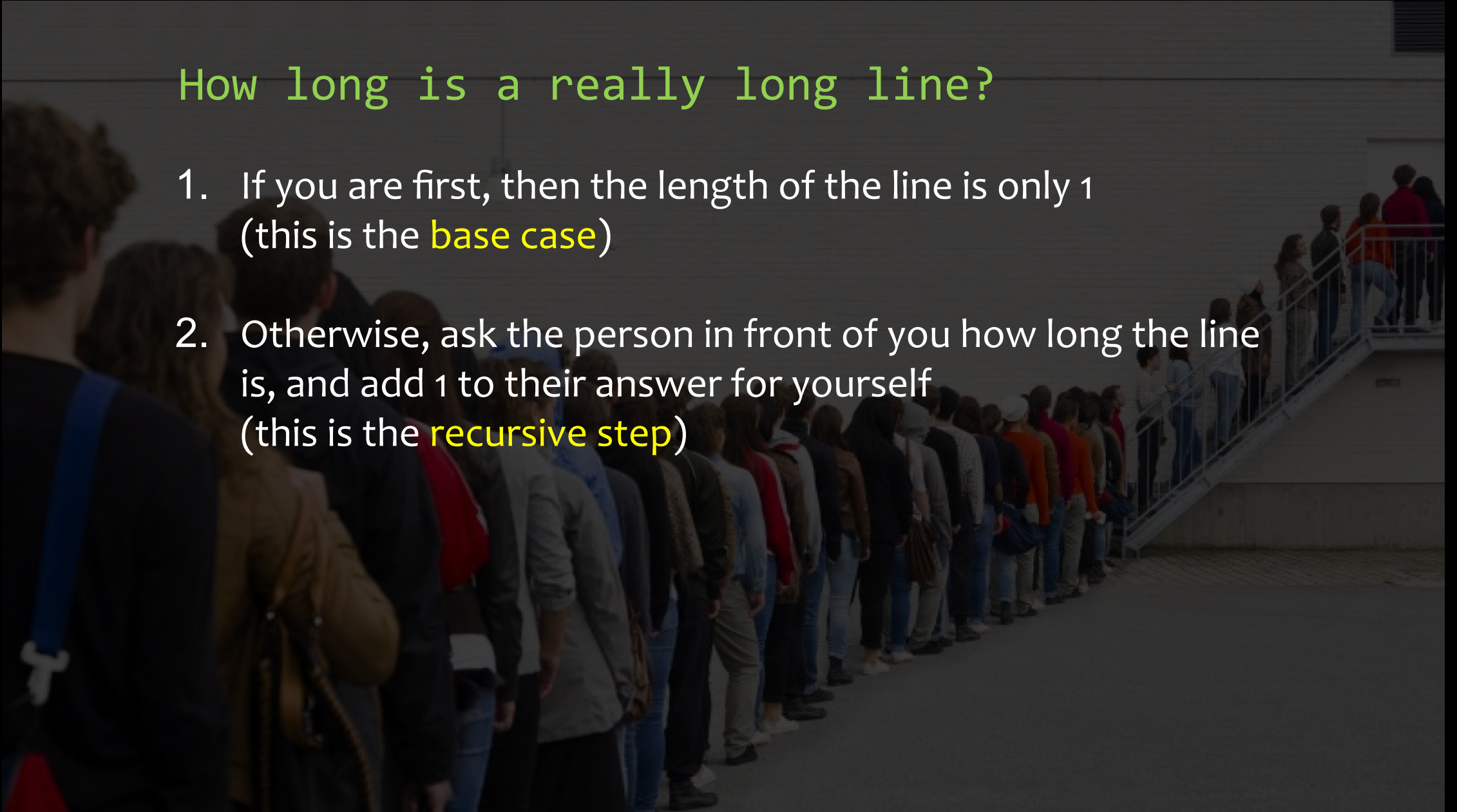
#beenHereSince5am
#amateurs



<https://www.inc.com/jill-krasny/why-waiting-in-lines-makes-things-more-appealing.html>

How long is a really long line?

1. If you are first, then the length of the line is only 1 (this is the **base case**)
2. Otherwise, ask the person in front of you how long the line is, and add 1 to their answer for yourself (this is the **recursive step**)



How to show everything in a folder and all of its subfolders

1. Show everything in folder
2. Use #1 on each subfolder



The file hierarchy is recursive!

<https://zapier.com/blog/organize-files-folders/>

Pseudocode to list all files in a folder

```
listAllFiles(folderOrFile):  
    if folderOrFile is file:  
        print folderOrFile  
    else: //now we know it's a folder instead  
        for each item in folderOrFile:  
            listAllFiles(item)
```


Pseudocode to list all files in a folder

```
listAllFiles(folderOrFile):
```

```
    if folderOrFile is file:
```

```
        print folderOrFile
```

```
    else: //now we know it's a folder instead
```

```
        for each item in folderOrFile:
```

```
            listAllFiles(item)
```

Base case:

When we see a file,
just print it



Pseudocode to list all files in a folder

```
listAllFiles(folderOrFile):  
    if folderOrFile is file:  
        print folderOrFile  
    else: //now we know it's a folder instead  
        for each item in folderOrFile:  
            listAllFiles(item)
```

Recursive step:
When we see a folder,
call the listAllFiles function on it
to list the files



search for a word in a dictionary

1. Base Case:

If there's no words in the dictionary then your word is not in it

1. Check the word in the middle of dictionary:

if it's your word then you're done!

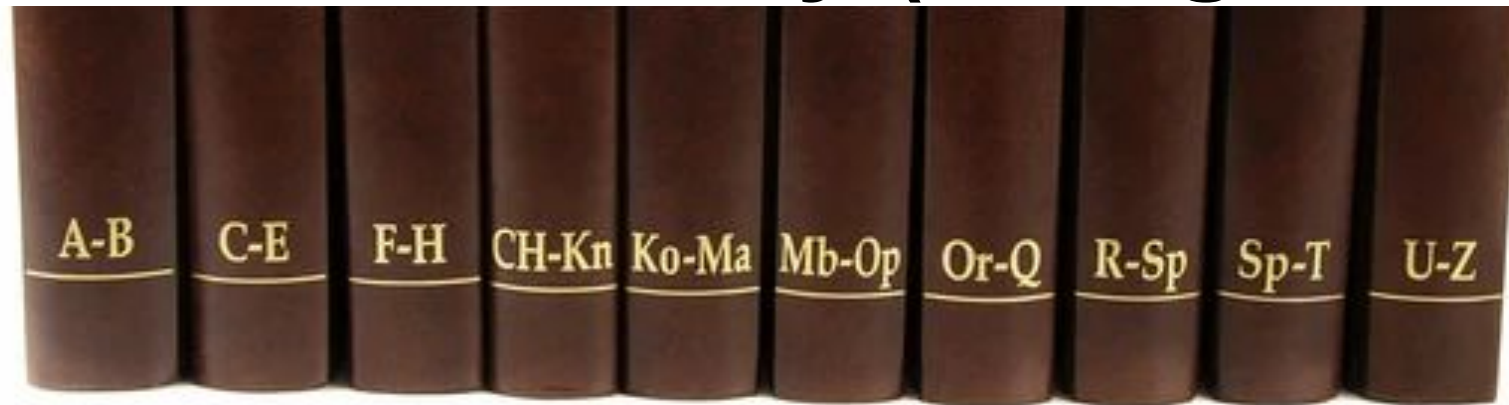
else

if your word is alphabetically earlier, **search** in left half

else **search** in right half

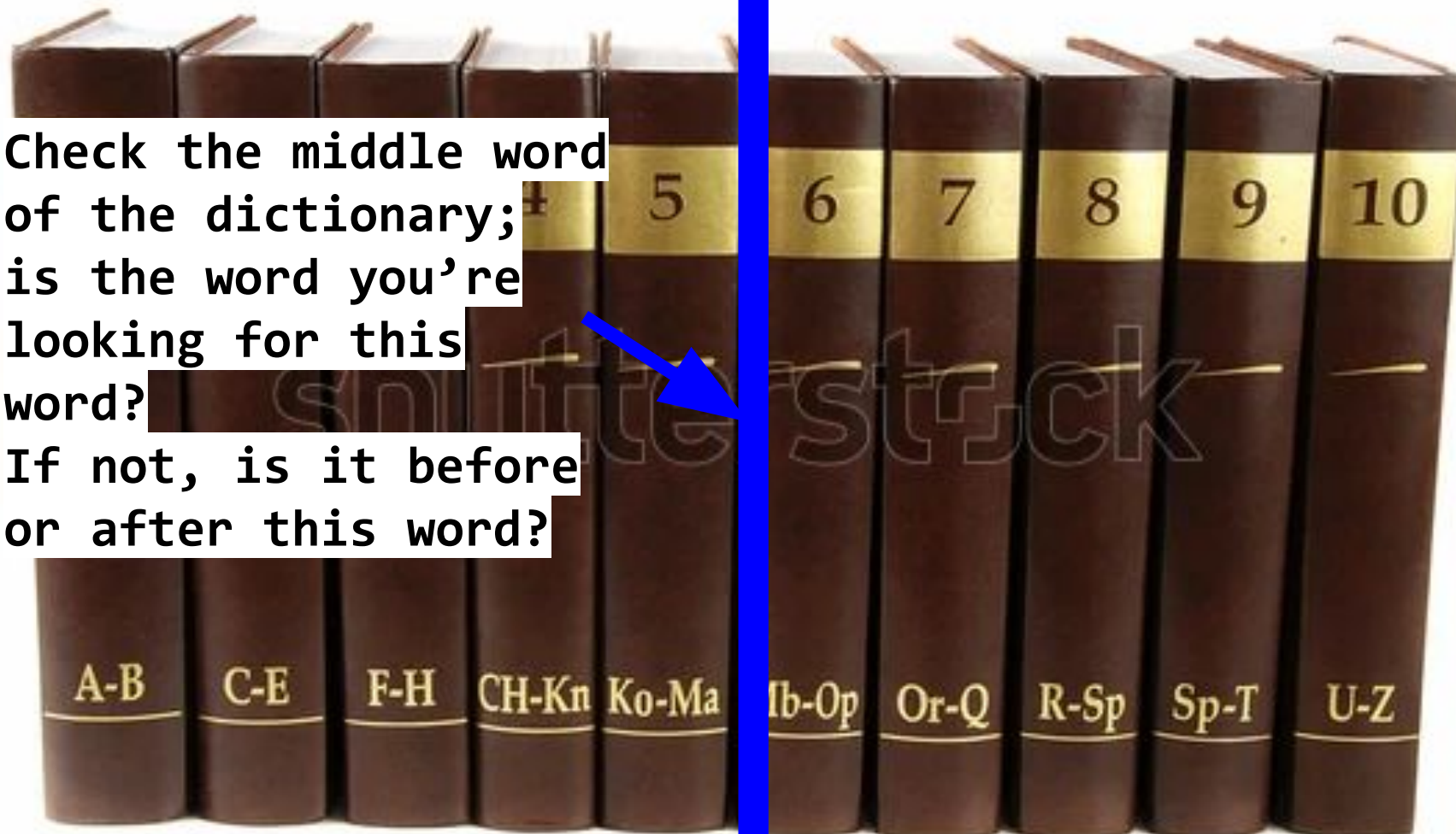


searchDictionary (String word)



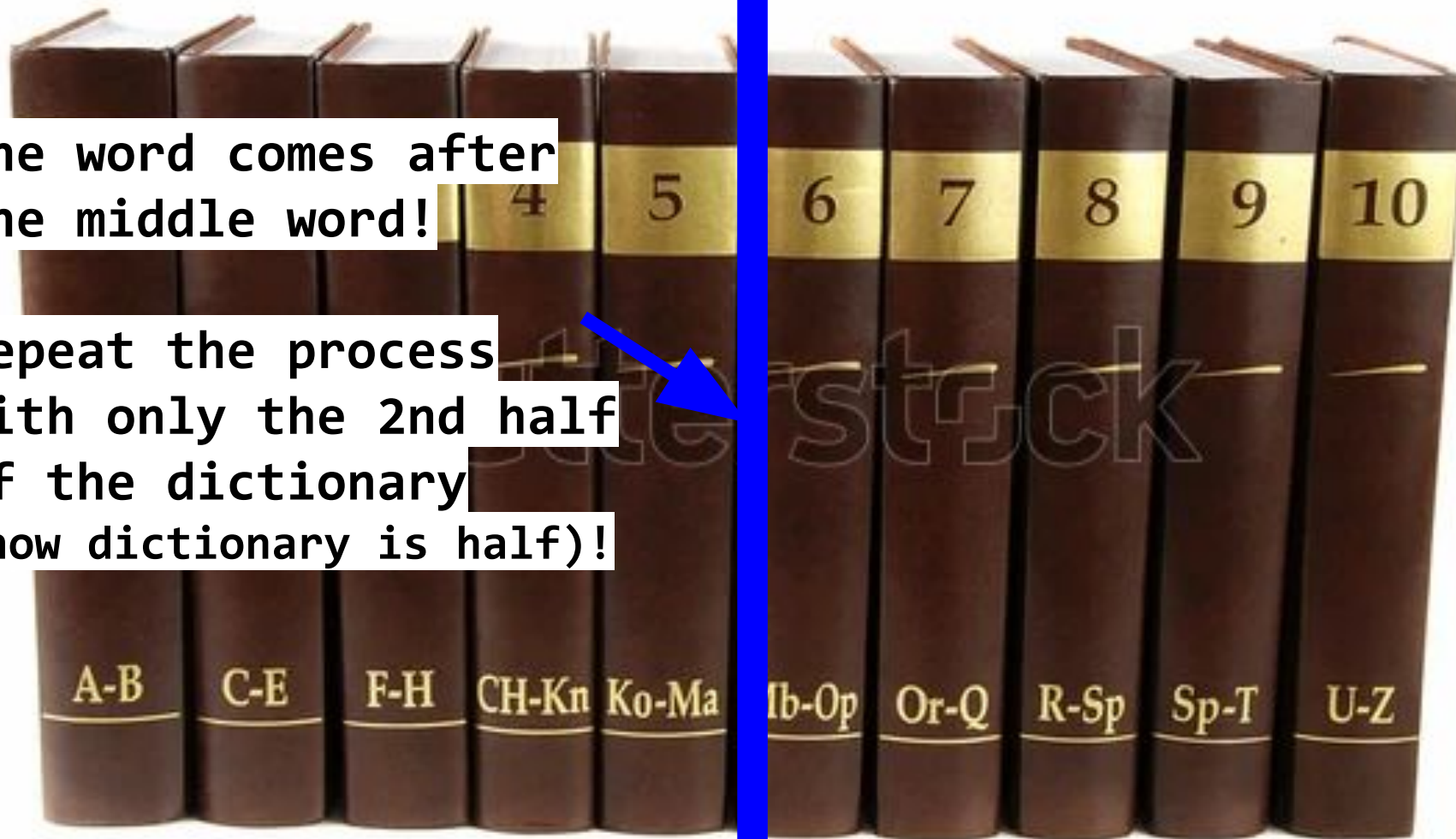
Check the middle word
of the dictionary;
is the word you're
looking for this
word?

If not, is it before
or after this word?



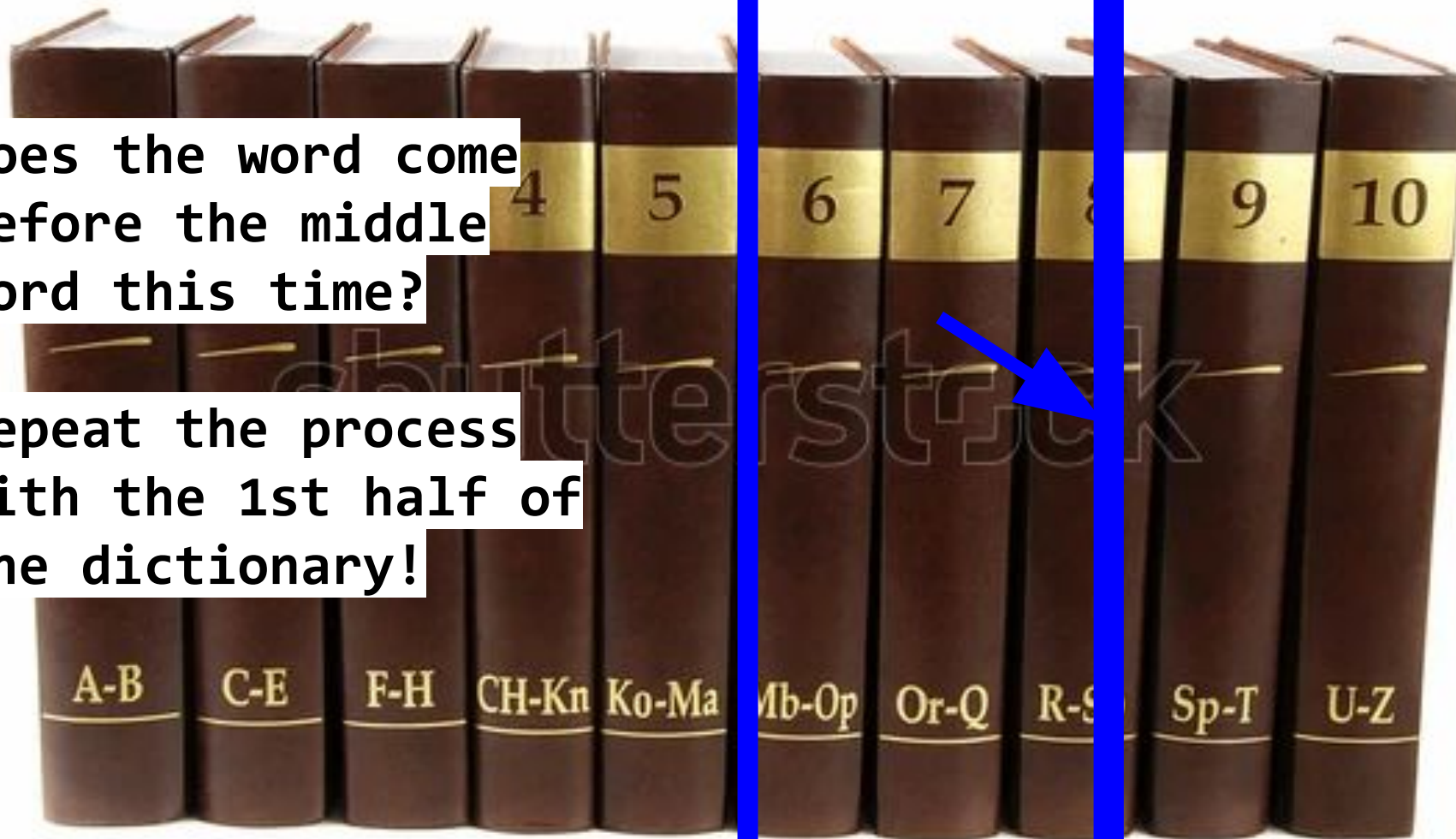
The word comes after
the middle word!

Repeat the process
with only the 2nd half
of the dictionary
(now dictionary is half)!



Does the word come
before the middle
word this time?

Repeat the process
with the 1st half of
the dictionary!



The word comes before
the middle word!

Repeat the process
with the 1st half of
the dictionary!



Other possible uses of recursion: Mathematical Summations

Sum all values from n to 1:

$\text{sumAll}(n) =$

$n + \text{sumAll}(n-1) =$

$n + ((n-1) + \text{sumAll}(n-2)) =$

...

$n + (n-1) + (n-2) + \dots + 2 + 1$

How do we implement recursion in Java?

- A. nested for loops
- B. a for loop in a while loop
- C. a function with a loop
- D. a function that calls itself
- E. lambda-R (the recursion operator)

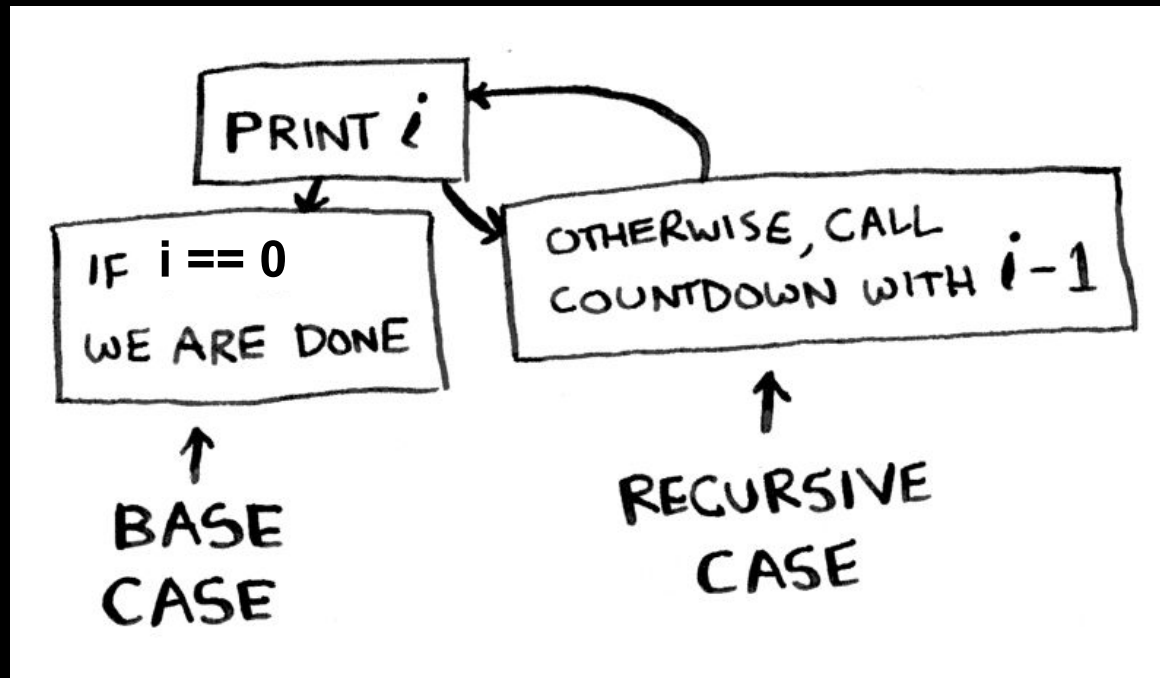
How do we implement recursion in Java?

- A. nested for loops
- B. a for loop in a while loop
- C. a function with a loop
- D. a function that calls itself**
- E. lambda-R (the recursion operator)

There is no lambda-R, the recursion operator. I made that up...

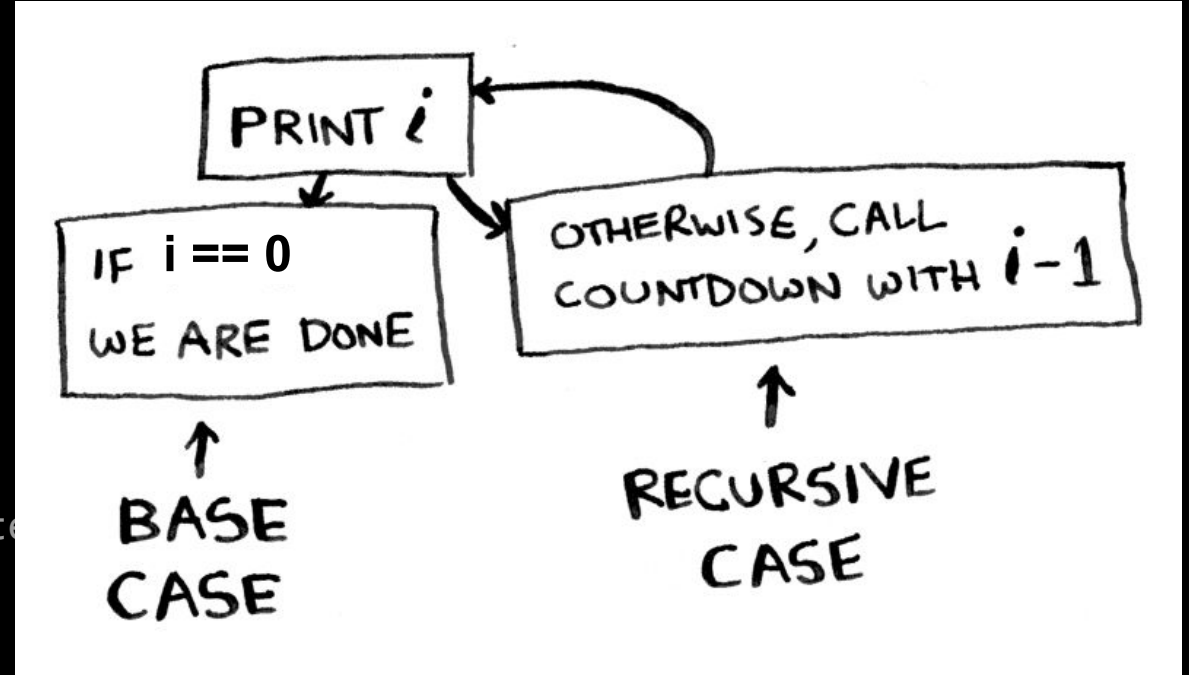
blastoff(int countdown)

1. If the countdown is zero, yell BLASTOFF!
2. Otherwise, yell the countdown, decrease the countdown by 1, and redo #1



blastoff(int countdown)

```
public static void blastoff(int countdown) {  
    if (countdown == 0) //base case  
        System.out.println("Blastoff!");  
    else {  
        System.out.println(countdown+"!");  
        blastoff(countdown-1); //recursive step  
    }  
}
```



<https://repl.it/@jspacco/Blastoff-Recursively>

Other possible uses of recursion: Factorials

Factorial of 5 =

`factorial(5) =`

`5 * factorial(4) =`

`5 * 4 * factorial(3) =`

`5 * 4 * 3 * factorial(2) =`

`5 * 4 * 3 * 2 * factorial(1) =`

`5 * 4 * 3 * 2 * 1`

Other possible uses of recursion

Factorial in math

$$5! = 5 * 4 * 3 * 2 * 1$$

A. $5! = 5 * 4!$

B. $4! = 4 * 3!$

C. $3! = 3 * 2!$

D. $2! = 2 * 1!$

E. $1! = 1$

Which is the base case?

Other possible uses of recursion

Factorial in math

$$5! = 5 * 4 * 3 * 2 * 1$$

A. $5! = 5 * 4!$

B. $4! = 4 * 3!$

C. $3! = 3 * 2!$

D. $2! = 2 * 1!$

E. $1! = 1$

Which is the base case?

`factorial(1)=1`

Other possible uses of recursion

Factorial in math

$$5! = 5 * 4 * 3 * 2 * 1$$

A. $5! = 5 * 4!$

B. $4! = 4 * 3!$

C. $3! = 3 * 2!$

D. $2! = 2 * 1!$

E. $1! = 1$

Which is the recursive step?

Other possible uses of recursion

Factorial in math

$$5! = 5 * 4 * 3 * 2 * 1$$

A. $5! = 5 * 4!$

B. $4! = 4 * 3!$

C. $3! = 3 * 2!$

D. $2! = 2 * 1!$

E. $1! = 1$

Which is the recursive step?

factorial in Java

```
static int fact(int n) {  
    // Base case  
    // Recursive step  
}
```

factorial(5) →
= 5 * factorial(4) →
= 5 * 4 * factorial(3) →
= 5 * 4 * 3 * factorial(2) →
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120

```
static int fact(int n) {  
    // Base case  
    // Recursive step  
}
```

factorial(5) 
= 5 * factorial(4) 
= 5 * 4 * factorial(3) 
= 5 * 4 * 3 * factorial(2) 
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120

Which is the best base case for calculating factorial?

A

```
if (n<=1) {  
    return 1;  
}
```

C

```
return fact(n-1);
```

B

```
return n-1;
```

D

```
return n * fact(n-1);
```

```
static int fact(int n) {  
    // Base case  
    // Recursive step  
}
```

$\text{factorial}(5)$
 $= 5 * \text{factorial}(4)$
 $= 5 * 4 * \text{factorial}(3)$
 $= 5 * 4 * 3 * \text{factorial}(2)$
 $= 5 * 4 * 3 * 2 * \text{factorial}(1)$
 $= 5 * 4 * 3 * 2 * 1$
 $= 120$

Which is the best base case for calculating factorial?

A

```
if (n<=1) {  
    return 1;  
}
```

C

```
return fact(n-1);
```

B

```
return n-1;
```

D

```
return n * fact(n-1);
```



```
static int fact(int n) {  
    if (n<=1) { //base case  
        return 1;  
    }  
    // Recursive step  
}
```

factorial(5) →
= 5 * factorial(4) →
= 5 * 4 * factorial(3) →
= 5 * 4 * 3 * factorial(2) →
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120



Which should be
the recursive step?

- A if (n >= 1) {
 return n;
}
- B return n * n-1;
- C return n * fact(n-1);
- D return fact(n) * fact(n-1);
- E none of the above

```
static int fact(int n) {  
    if (n<=1) { //base case  
        return 1;  
    }  
    // Recursive step  
}
```

factorial(5) →
= 5 * factorial(4) →
= 5 * 4 * factorial(3) →
= 5 * 4 * 3 * factorial(2) →
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120



Which should be
the recursive step?

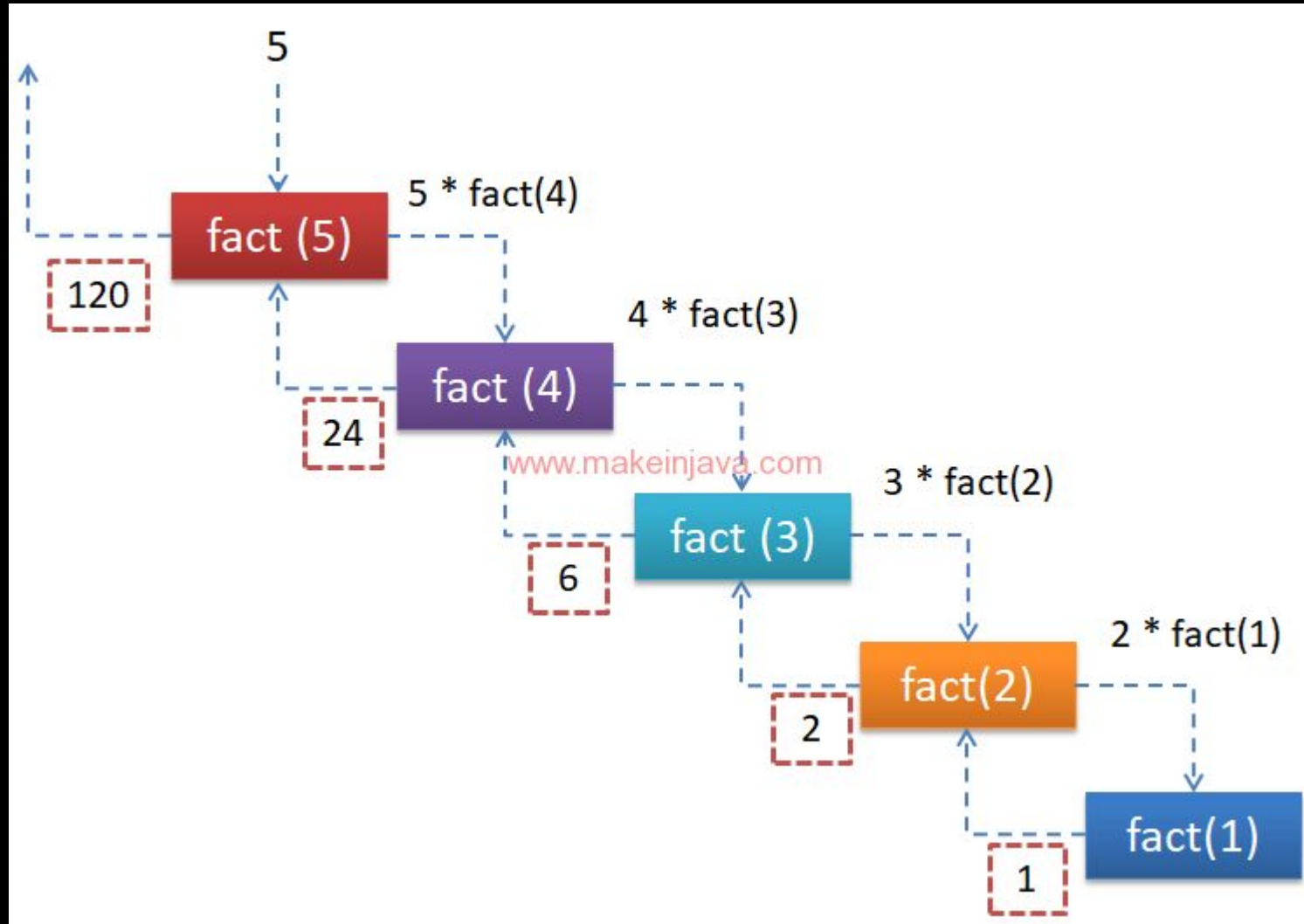
- A if (n >= 1) {
 return n;
}
- B return n * n-1;
- C return n * fact(n-1);**
- D return fact(n) * fact(n-1);
- E none of the above

Recursion can't go on forever.
Eventually we get a call
where $n \leq 1 \leftarrow$ BASE CASE

```
static int fact(int n) {  
    if (n <= 1) { //base case  
        return 1;  
    }  
    return n * fact(n-1); //recursive step  
}
```

$$\begin{aligned} &\text{factorial}(5) \curvearrowright \\ &= 5 * \text{factorial}(4) \curvearrowright \\ &= 5 * 4 * \text{factorial}(3) \curvearrowright \\ &= 5 * 4 * 3 * \text{factorial}(2) \curvearrowright \\ &= 5 * 4 * 3 * 2 * \text{factorial}(1) \\ &= 5 * 4 * 3 * 2 * 1 \\ &= 120 \end{aligned}$$

<https://repl.it/@jspacco/141-factorial>



What happens if we miss
the base case?

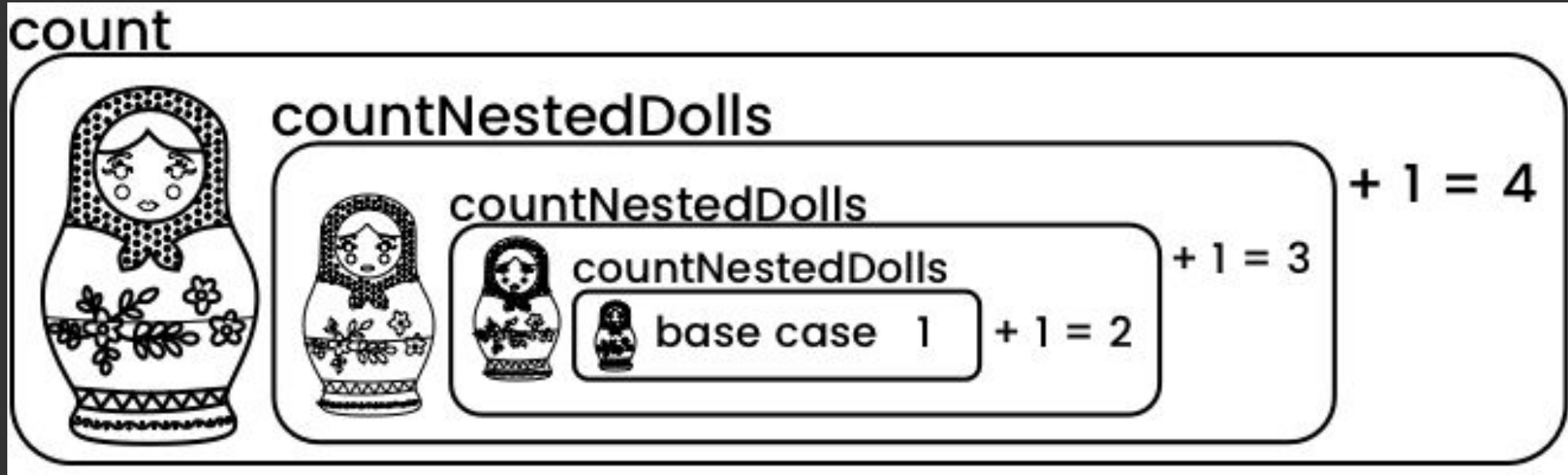
INFINITE RECURSION

```
static int fact(int n) {  
    if (n<=1) { //base case  
        return 1;  
    }  
    return n * fact(n-1); //recursive step  
}
```



<https://replit.com/@VeraKazakova1/141-factorial-1#Main.java>

Quick recap of recursive solutions:



Establish **base case**:

when problem is very simple and we immediately know the answer

Establish **recursive step**:

solve a tiny piece and combine with remaining problem

Example:

find sum of first n positive integers

$n + (n-1) + (n-2) + \dots + 3 + 2 + 1$

base case:

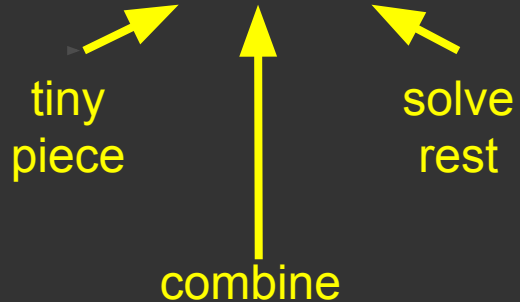
//when problem is very simple and we immediately know the answer

```
if (n==1)
    return 1;
```

recursive step:

//solve a tiny piece and combine with remaining problem

```
return n + sum (n-1);
```



Example:

find sum of first n positive integers

$n + (n-1) + (n-2) + \dots + 3 + 2 + 1$

```
public static int sumFirstNPositive(int n){  
    if (n==1) // base case  
        return 1;  
    return n + sum (n-1); //recursive step  
}
```

Towers of Hanoi code

```
public class Towers {  
  
    public void move(int n, int from, int to, int extra) {  
        if(n == 0)  
            return;  
  
        move(n-1, from, extra, to);  
        System.out.println("Move disk " + n + " from peg " + from +  
                           " to peg " + to);  
        move(n-1, extra, to, from);  
    }  
}
```

The Fibonacci numbers

The first two are $F_0=0$ and $F_1=1$.

(some people start with 1 and 1, leading to the same overall sequence of values)

Then each new number is the sum of the previous two numbers.

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

How would we calculate the n-th Fibonacci number RECURSIVELY?

The parameter should be the n-th number of the sequence:

fib(0) should return 0

fib(1) should return 1

fib(2) should return 1

etc.

```
public static int fib(int n) { //Fibonacci
    // base case?
}
```

A: `return fib(n) + fib(n-1)`

B: `if (n<2) {
 return fib(n-1) + fib(n-2);
}`

C: `if (n<2) {
 return n;
}`

D: `if (n==0)
 return 0;
if (n==1)
 return 1;`

E: none of the above

```
public static int fib(int n) { //Fibonacci
    // base case?
}
```

A: `return fib(n) + fib(n-1)`

B: `if (n<2) {
 return fib(n-1) + fib(n-2);
}`

C: `if (n<2) {
 return n;
}`

D: `if (n==0)
 return 0;
if (n==1)
 return 1;`

E: none of the above

Multiple base cases are totally fine!
Have as many as you want,
but at least one,
or else we'll have an infinite loop...




```
public static int fib(int n) { //Fibonacci
    if (n<2) // base case
        return 1;
    // recursive step?
}
```

A: return fib(n) + fib(n-1);

B: return fib(n-1);

C: int count=0;
for (int i=0; i<n-1; i++) {
 count+=fib(i);
}
return count;

D: return fib(n-1) + fib(n-2);

E: none of the above

```
public static int fib(int n) { //Fibonacci
    if (n<2) // base case
        return 1;
    // recursive step?
}
```

A: return fib(n) + fib(n-1);

B: return fib(n-1);

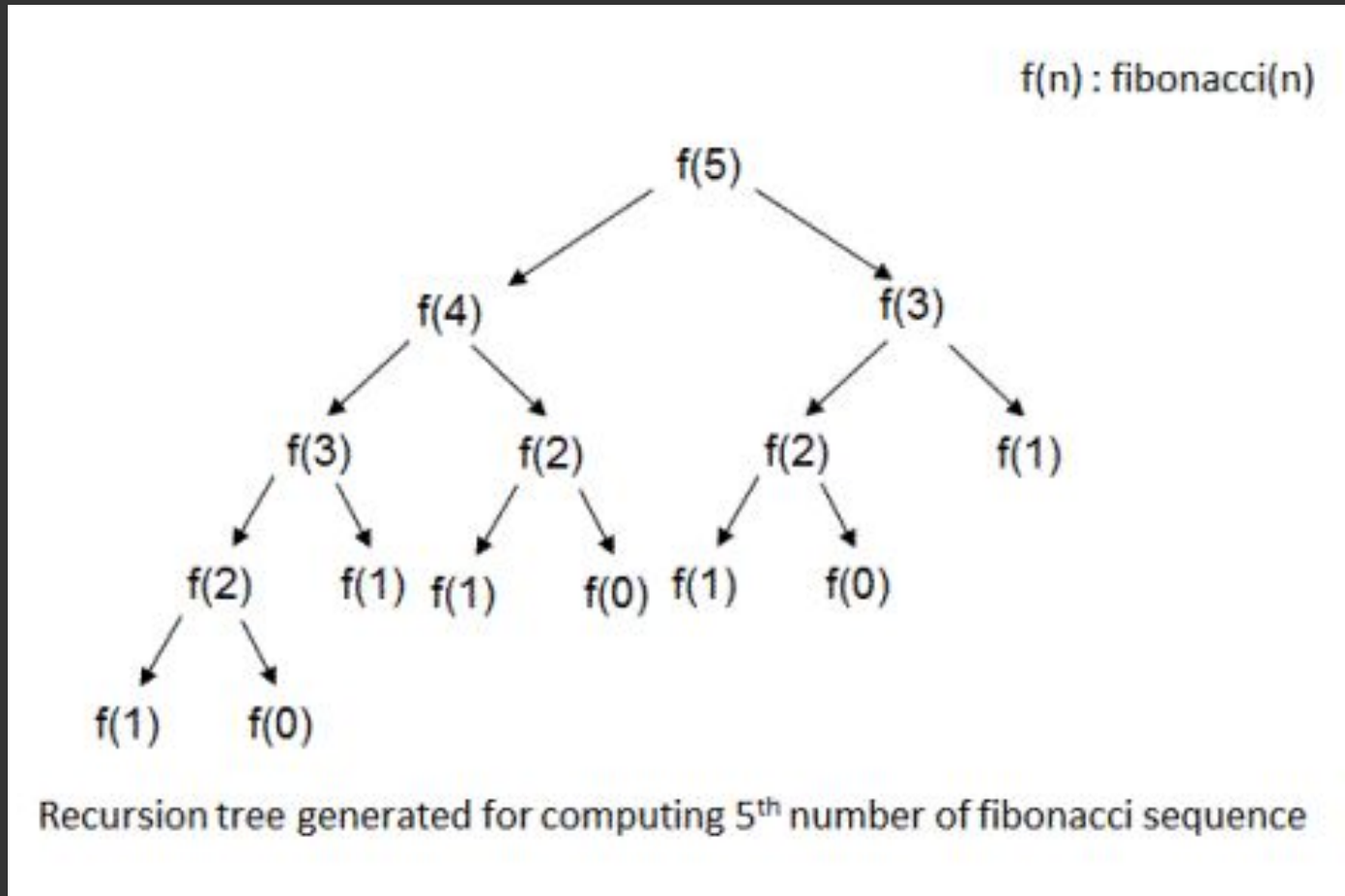
C: int count=0;
for (int i=0; i<n-1; i++) {
 count+=fib(i);
}
return count;

D: return fib(n-1) + fib(n-2);

E: none of the above

Fibonacci

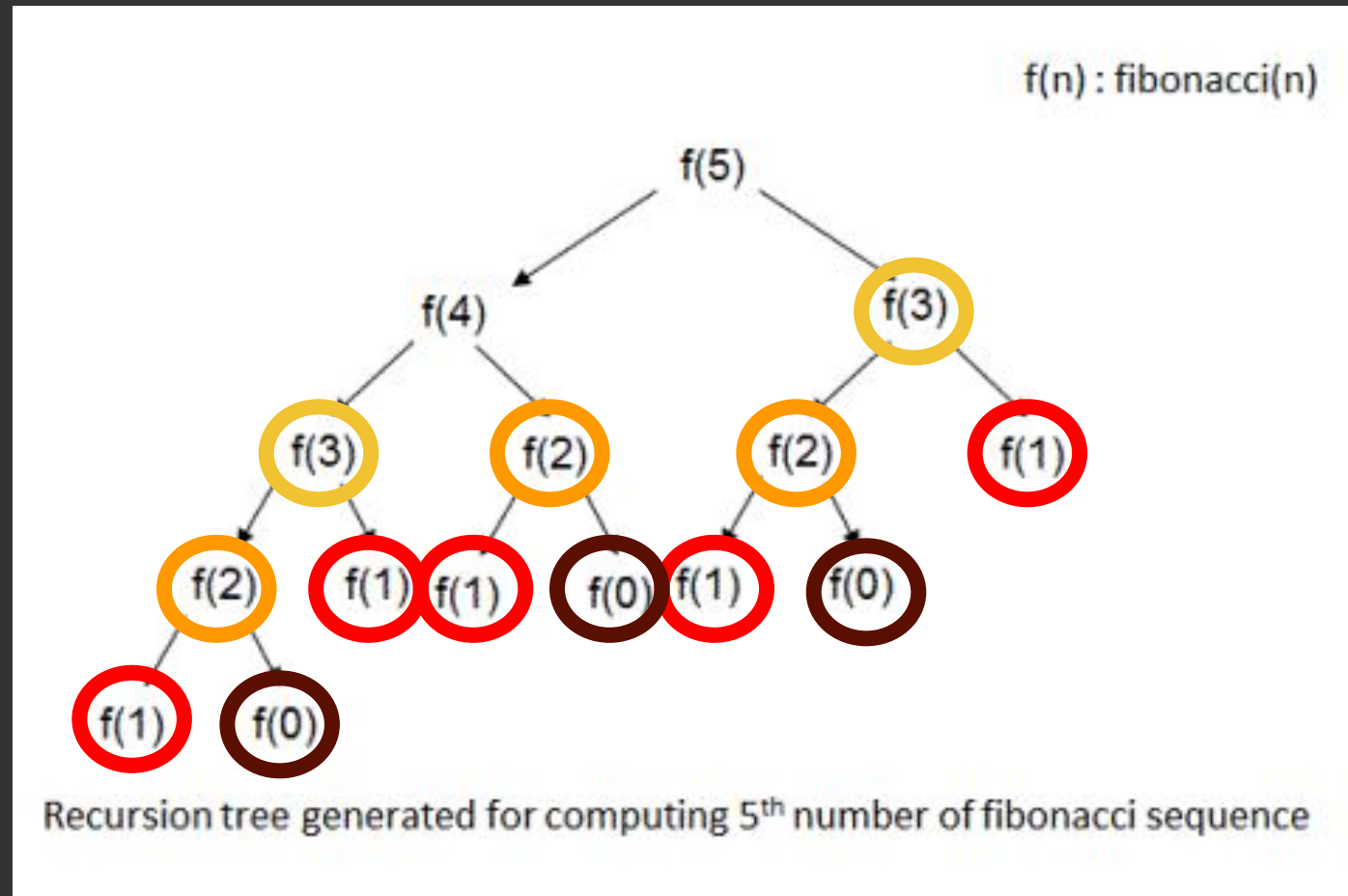
```
static int fib(int n) {  
    if (n<2) {  
        return n;  
    }  
  
    return fib(n-1) + fib(n-2);  
}
```



Fibonacci

```
static int fib(int n) {  
    if (n<2) {  
        return n;  
    }  
  
    return fib(n-1) + fib(n-2);  
}
```

Plot twist:
recursive fibonacci is
actually really inefficient.

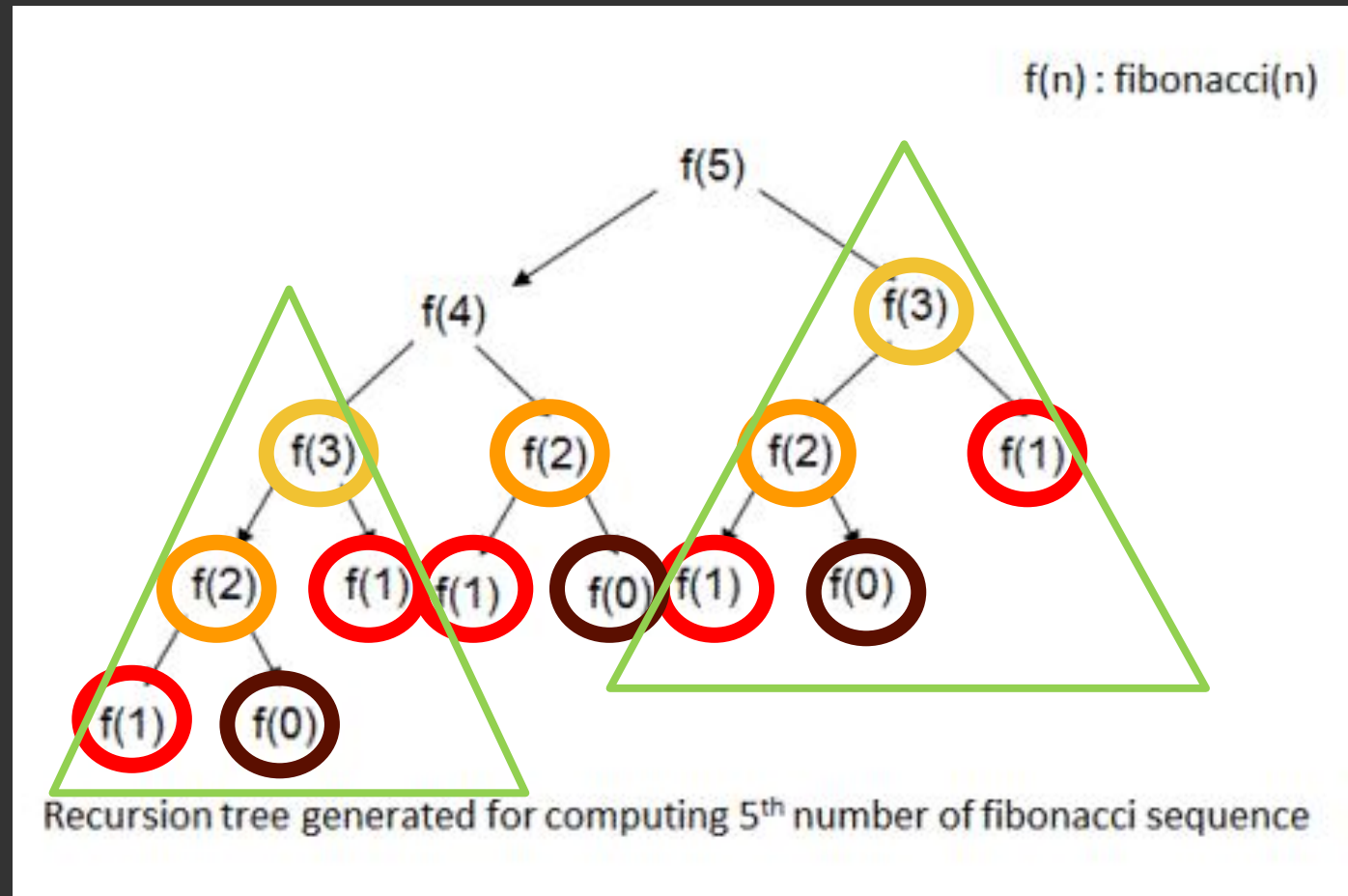


Fibonacci

```
static int fib(int n) {  
    if (n<2) {  
        return 1;  
    }  
  
    return fib(n-1) + fib(n-2);  
}
```

Plot twist:
recursive fibonacci is
actually really inefficient.

Solution: 1) memoization or 2) dynamic programming
(basically just remember solutions to subproblems you already solved)



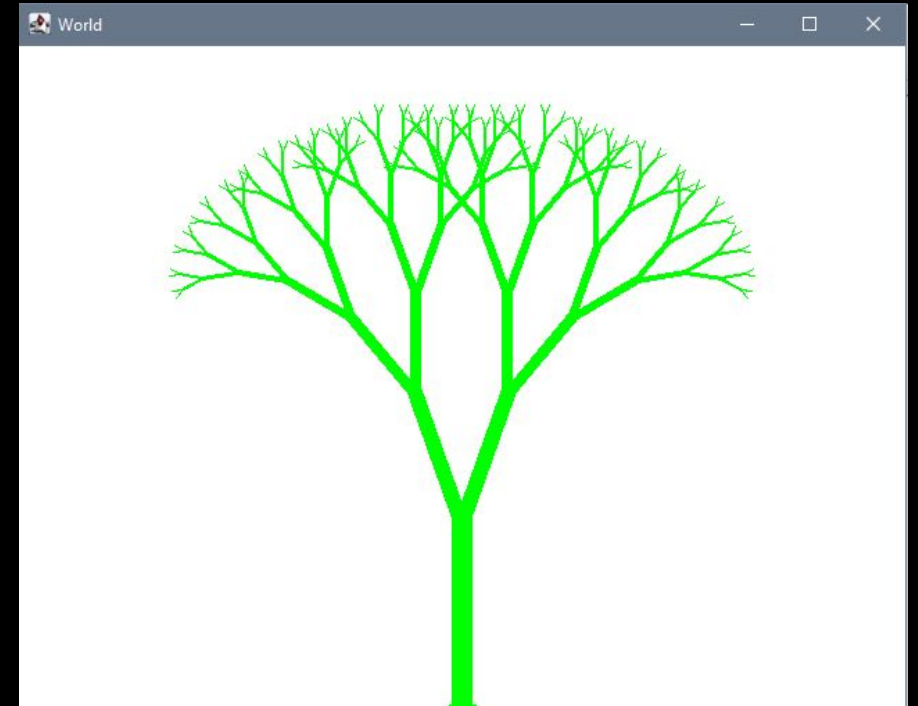
Fractal Trees!

- <https://www.visnos.com/demos/fractal>
- <http://mwskirpan.com/FractalTree/>
- <https://tarokuriyama.com/fractaltree/>

Let's make a Turtle draw recursively

```
public static void tree(Turtle t, int width) {  
    //base case  
    //if width<=1, do nothing (i.e. return)  
    //recursive step  
    if (width >= 1){  
        t.setPenWidth(width);  
        t.forward(width * 9);  
        t.turn(20); //turn to right of center  
        tree(t, width * 3/4);  
        t.turn(-40); //turn to left of center  
        tree(t, width * 3/4);  
        t.turn(20); //go back to the center  
        t.backward(width * 9); //move turtle back  
        t.setPenWidth(width); //reset pen size  
    }  
}
```

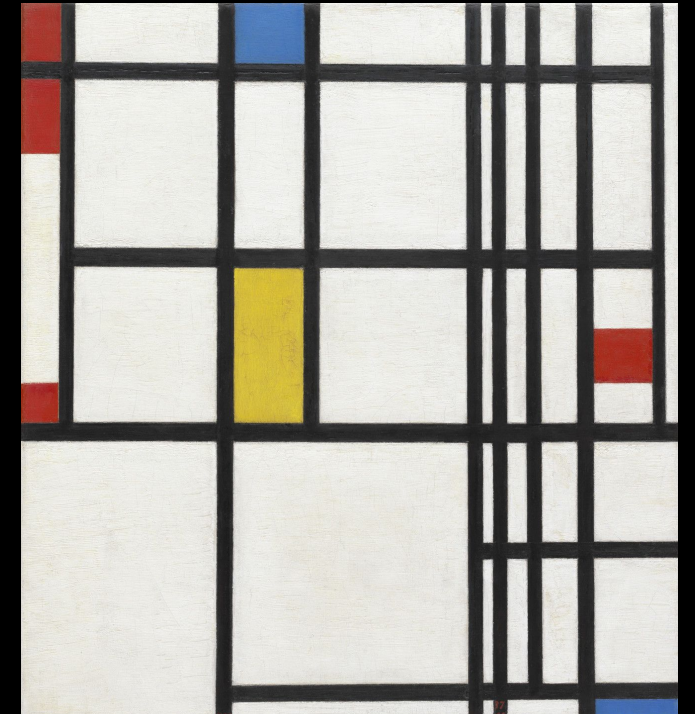
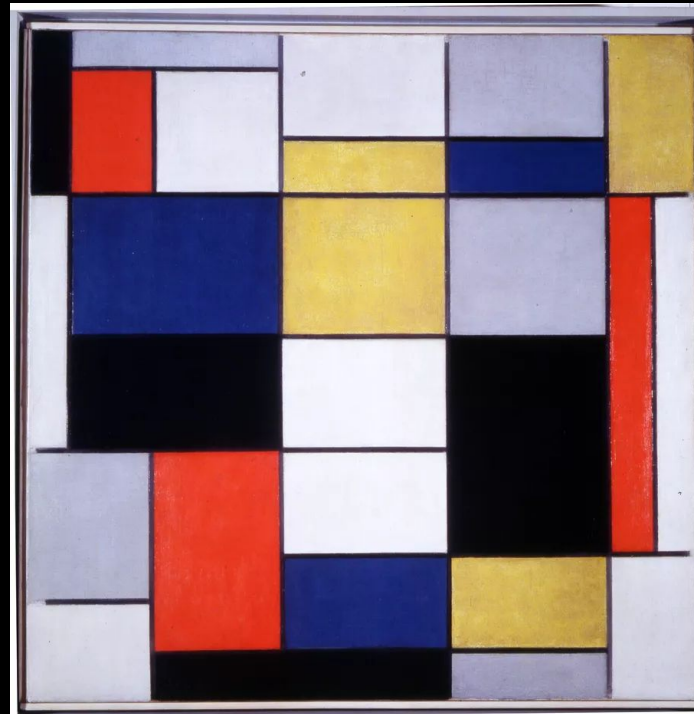
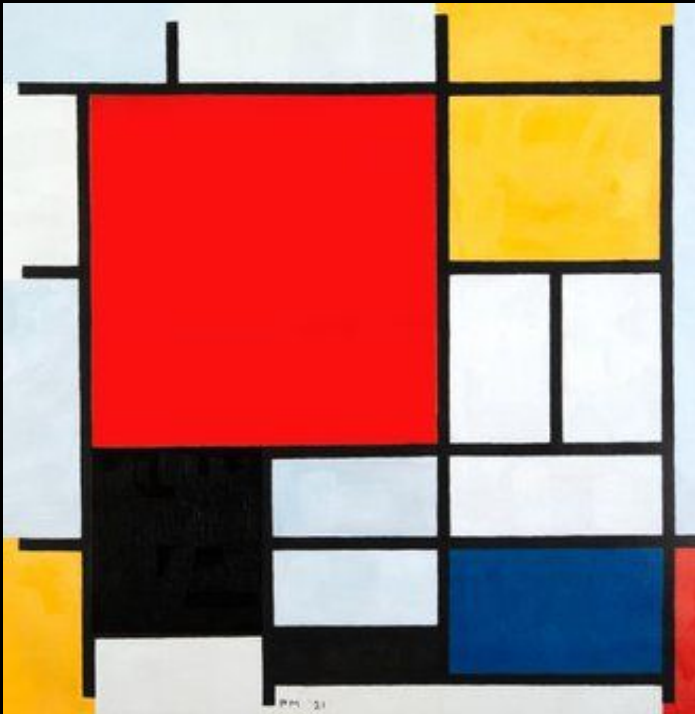
```
public static void main(String[] args) {  
    Turtle t = new Turtle(new World());  
    t.penUp();  
    t.backward(400);  
    t.penDown();  
    tree(t, 15);  
}
```



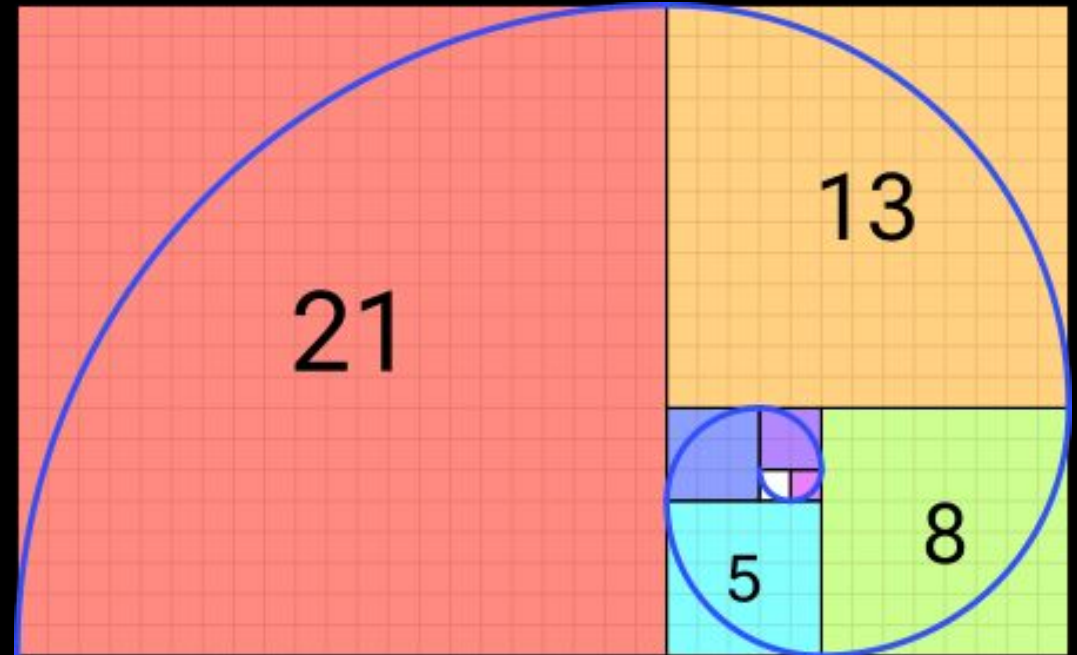
Mondrian

http://en.wikipedia.org/wiki/Piet_Mondrian

<http://daveho.github.io/mondrian/>



We can also tile/draw with Fibonacci!





We must go one level deeper



And then what?



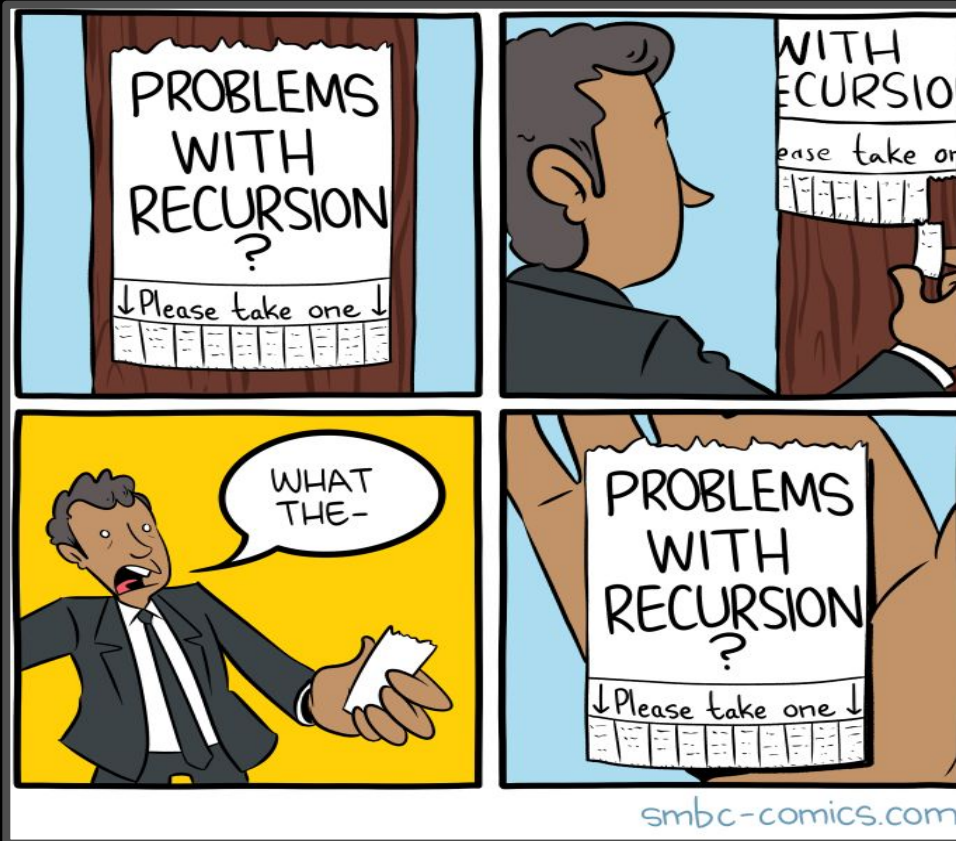
We must go one level deeper



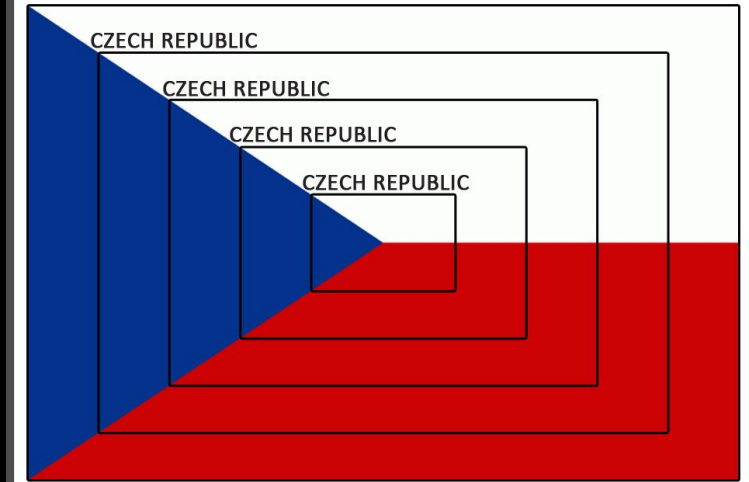
And then what?

RECURSION

To understand
recursion,
one must first
understand
recursion



Flag of Czech Republic



WHAT ARE YOU WORKING ON?

TRYING TO FIX THE PROBLEMS I
CREATED WHEN I TRIED TO FIX
THE PROBLEMS I CREATED WHEN
I TRIED TO FIX THE PROBLEMS
I CREATED WHEN...



Codingbat: [Recursion-1.noX](#): Given a string, compute recursively a new string where all the 'x' chars have been removed.

When using the string length as the problem size, which of the following is the base case:

- A. The string is empty
- B. The string has a single character
- C. The string doesn't contain 'x'
- D. The string contains only one 'x'
- E. Not exactly one of the above

Codingbat: Recursion-1.noX: Given a string, compute recursively a new string where all the 'x' chars have been removed.

When using the string length as the problem size, which of the following is the base case:

- A. The string is empty //Best answer
- B. The string has a single character //Doesn't work for empty string
- C. The string doesn't contain 'x' //Can also be made to work
- D. The string contains only one 'x'
- E. Not exactly one of the above

Some recursive problems (let's solve them together)

- noX
- changeXY
- countHi
- allStar
- endX

Recursion-1 > noX

[prev](#) | [next](#) | [chance](#)

Given a string, compute recursively a new string where all the 'x' chars have been removed.

noX("xaxb") → "ab"
noX("abc") → "abc"
noX("xx") → ""

Go

...Save, Compile, Run (ctrl-enter)

```
public String noX(String str) {  
    //base case  
    if (str.length()==0)  
        return str; //""  
  
    //-----recursive step-----  
    //if x --> remove it  
    if (str.charAt(0)=='x'){  
        return noX(str.substring(1));  
    }  
    //if not x --> keep it  
    return str.charAt(0)+noX(str.substring(1));  
}
```


Recursion-1 > changeXY

[prev](#) | [next](#) | [chance](#)

Given a string, compute recursively (no loops) a new string where all the lowercase 'x' chars have been changed to 'y' chars.

changeXY("codex") → "codey"

changeXY("xxhixx") → "yyhiyy"

changeXY("xhixhix") → "yhiyhiy"

Go

...Save, Compile, Run (ctrl-enter)

```
public String changeXY(String str) {  
    //base case: empty string  
    if (str.length()==0)  
        return "";  
    //recursive step  
    //if I'm looking at an x, keep a y here and solve rest  
    if (str.charAt(0)=='x')  
        return 'y'+ changeXY(str.substring(1));  
    //otherwise keep current letter here and solve the rest  
    return str.charAt(0) + changeXY(str.substring(1));  
}
```

Recursion-1 > countHi

[prev](#) | [next](#) | [chance](#)

Given a string, compute recursively (no loops) the number of times lowercase "hi" appears in the string.

countHi("xxhixx") → 1

countHi("xhixhix") → 2

countHi("hi") → 1

Go

...Save, Compile, Run (ctrl-enter)

```
public int countHi(String str) {  
  
    // base case: string less than 2 chars  
    if (str.length() < 2)  
        return 0;  
  
    // recursive step: we know we have at least 2 chars  
    // if we are looking at hi currently --> skip both, count 1 + solve rest  
    if (str.charAt(0) == 'h' && str.charAt(1) == 'i')  
        return 1 + countHi(str.substring(2));  
    // not looking at hi currently  
    return 0 + countHi(str.substring(1));  
}
```

Recursion-1 > allStar

[prev](#) | [next](#) | [chance](#)

Given a string, compute recursively a new string where all the adjacent chars are now separated by a "*".

allStar("hello") → "h*e*l*l*o"

allStar("abc") → "a*b*c"

allStar("ab") → "a*b"

Go

...Save, Compile, Run (ctrl-enter)

```
public String allStar(String str) {  
    //base case: no chars left to separate by * (i.e. only 1 or no chars)  
    if (str.length() < 2)  
        return str;  
    //recursive step: keep first char, add asterisk, and solve rest  
    return "" + str.charAt(0) + '*' + allStar(str.substring(1));  
    //note: char+char gets turned into int+int;  
    //if we want a string, we need to ensure compiler understands + as concatenation  
    //for this, start with an empty string before concatenating your characters  
}
```


Recursion-1 > endX

[prev](#) | [next](#) | [chance](#)

Given a string, compute recursively a new string where all the lowercase 'x' chars have been moved to the end of the string.

endX("xxre") → "rexx"

endX("xxhixx") → "hixxxx"

endX("xhixhix") → "hihixxx"

Go

...Save, Compile, Run (ctrl-enter)

```
public String endX(String str) {  
    //base case: empty string  
    if (str.length()==0)  
        return "";  
  
    //recursive step  
    //if it's an x, solve rest and concatenate with x  
    if (str.charAt(0)=='x')  
        return endX(str.substring(1)) + 'x';  
    //if not x, then keep where it is and solve rest  
    return str.charAt(0) + endX(str.substring(1));  
}
```

Done for Now

More recursive art

<http://www.cis.upenn.edu/~cis110/13sp/hw/hw03/gallery.shtml>